

COMPUTER NETWORKS

LAB ASSESSMENT - 3

SUBMITTED BY:
APOORVA KARNWAL (22BCE0756)

QUESTION1 STOP AND WAIT PROTOCOL:

SERVER SIDE CODE:

```
import socket
import time
HOST = '10.30.157.11'
PORT = 65432, i=0
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server listening on {HOST}:{PORT}")
    conn, addr = server_socket.accept()
    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode()
            print(f"Received: {message}")
            i=i+1
            if i==2:
                time.sleep(5)
                print("sleep")
            conn.sendall(b'ACK')
            print(f"ACK sent to client for {message}")
```

a.py - C:/Users/batch1/AppData/Local/Programs/Python/Python311/a.py (3.11.3)

File Edit Format Run Options Window Help

```
import socket
import time
HOST = '10.30.157.11'
PORT = 65432
i=0

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server listening on {HOST}:{PORT}")

    conn, addr = server_socket.accept()
    with conn:
        print(f"Connected by {addr}")

        while True:

            data = conn.recv(1024)
            if not data:
                break

            message = data.decode()
            print(f"Received: {message}")
            i=i+1

            if i==2:
                time.sleep(5)
                print("sleep")

            conn.sendall(b'ACK')
            print(f"ACK sent to client for {message}")
```

OUTPUT:

```
>>> ===== RESTART: C:/Users/batch1/AppData/Local/Programs
Server listening on 10.30.157.11:65432
Connected by ('10.30.157.13', 1650)
Received: Packet 0
ACK sent to client for Packet 0
Received: Packet 1
sleep
ACK sent to client for Packet 1
Received: Packet 1
ACK sent to client for Packet 1
Received: Packet 2
ACK sent to client for Packet 2
```

CLIENT CODE:

```
import socket
import time
HOST = '10.30.157.11'
PORT = 65432
messages = ['Packet 0', 'Packet 1', 'Packet 2']
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    client_socket.connect((HOST, PORT))
    client_socket.settimeout(3.0)
    for message in messages:
        while True:
            try:
                client_socket.sendall(message.encode())
                print(f"Sent: {message}")
                ack = client_socket.recv(1024).decode()
                if ack == 'ACK':
                    print(f"Received ACK for: {message}")
                    break
            except socket.timeout:
                print(f"Timeout! Resending: {message}")
                time.sleep(1)

import socket

import time

HOST = '10.30.157.11'

PORT = 65432

messages = ['Packet 0', 'Packet 1', 'Packet 2']

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    |
        client_socket.connect((HOST, PORT))

        client_socket.settimeout(3.0)

        for message in messages:

            while True:

                try:

                    client_socket.sendall(message.encode())

                    print(f"Sent: {message}")

                    ack = client_socket.recv(1024).decode()

                    if ack == 'ACK':

                        print(f"Received ACK for: {message}")

                        break

                except socket.timeout:

                    print(f"Timeout! Resending: {message}")

                    time.sleep(1)
```

OUTPUT:

```
>>> == RESTART: C:\Users\batch1\AppData\Local\Programs\Python\Python311\client2.py =
Sent: Packet 0
Received ACK for: Packet 0
Sent: Packet 1
Timeout! Resending: Packet 1
Sent: Packet 1
Received ACK for: Packet 1
Sent: Packet 2
Received ACK for: Packet 2
>>>
```

QUESTION 2: GO BACK N ARQ

SERVER SIDE CODE:

```
import socket
import random
def receive_frames(conn, N):
    buffer = ""
    while True:
        try:
            data = conn.recv(1024).decode()
            if not data:
                break
            buffer += data

            frames = buffer.split('\n')
            for frame in frames[:-1]:
                if frame:
                    print(f"Received {frame.strip()}")
                    frame_num = int(frame.split()[1])
                    if random.randint(1, 10) <= 9:
                        ack_msg = f"ACK {frame_num}"
                        print(f"Sending acknowledgment for Frame {frame_num}")
                        conn.send(ack_msg.encode())
                    else:
                        print(f"Packet Loss {frame_num}")
                        nack_msg = f"NACK {frame_num}"
                        conn.send(nack_msg.encode())
                buffer = frames[-1]

        except Exception as e:
            print(f"Error: {e}")
            break
def main():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('172.16.44.112', 9999))
    server_socket.listen(1)
    print("Waiting for client connection...")
    conn, addr = server_socket.accept()
    print(f"Connected to {addr}")
    N = int(input("Enter the Window Size: "))
    receive_frames(conn, N)
    conn.close()
    server_socket.close()

if __name__ == "__main__":
    main()
```

CODE SCREENSHOT

```
server.py > receive_frames
1  import socket
2  import random
3  def receive_frames(conn, N):
4      buffer = ""
5      while True:
6          try:
7              data = conn.recv(1024).decode()
8              if not data:
9                  break
10             buffer += data
11
12             frames = buffer.split('\n')
13             for frame in frames[:-1]:
14                 if frame:
15                     print(f"Received {frame.strip()}")
16                     frame_num = int(frame.split()[1])
17                     if random.randint(1, 10) <= 9:
18                         ack_msg = f"ACK {frame_num}"
19                         print(f"Sending acknowledgment for Frame {frame_num}")
20                         conn.send(ack_msg.encode())
21                     else:
22                         print(f"Packet Loss {frame_num}")
23                         nack_msg = f"NACK {frame_num}"
24                         conn.send(nack_msg.encode())
25                 buffer = frames[-1]
26
27             except Exception as e:
28                 print(f"Error: {e}")
29                 break
30
31 def main():
32     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
33     server_socket.bind(('172.16.44.112', 9999))
34     server_socket.listen(1)
35     print("Waiting for client connection...")
36     conn, addr = server_socket.accept()
37     print(f"Connected to {addr}")
38     N = int(input("Enter the Window Size: "))
39     receive_frames(conn, N)
40     conn.close()
41     server_socket.close()
42
43 if __name__ == "__main__":
44     main()
```

OUTPUT

▼ TERMINAL

 Code + ▼   .

```
lpoorva23/Desktop/hack/server.py"
Waiting for client connection...
Connected to ('172.16.44.112', 55683)
Enter the Window Size: 3
Received Frame 1
Sending acknowledgment for Frame 1
Received Frame 2
Sending acknowledgment for Frame 2
Received Frame 3
Sending acknowledgment for Frame 3
Received Frame 2
Packet Loss 2
Received Frame 3
Sending acknowledgment for Frame 3
Received Frame 4
Sending acknowledgment for Frame 4
Received Frame 2
Sending acknowledgment for Frame 2
Received Frame 3
Sending acknowledgment for Frame 3
Received Frame 4
Sending acknowledgment for Frame 4
Received Frame 2
Sending acknowledgment for Frame 2
Received Frame 3
Sending acknowledgment for Frame 3
Received Frame 4
Sending acknowledgment for Frame 4
Received Frame 5
Sending acknowledgment for Frame 5
Received Frame 6
Sending acknowledgment for Frame 6
Received Frame 5
Sending acknowledgment for Frame 5
Received Frame 6
Sending acknowledgment for Frame 6
Received Frame 5
Sending acknowledgment for Frame 5
Received Frame 6
Sending acknowledgment for Frame 6
Received Frame 6
Packet Loss 6
Received Frame 6
Sending acknowledgment for Frame 6
```

CLIENT SIDE CODE:

```
import socket
import random
def transmission(client_socket, N, tf):
    i = 1

    tt = 0
    while i <= tf:
        z = 0

        try:

            for k in range(i, min(i + N, tf + 1)):
                msg = f"Frame {k}\n"
                client_socket.send(msg.encode())
                print(f"Sending {msg.strip()}...")
                tt += 1
            for k in range(i, min(i + N, tf + 1)):
                ack = client_socket.recv(1024).decode()
                if ack == f"ACK {k}":
                    print(f"Acknowledgment received for Frame {k}")
                    z += 1
                else:
                    print(f"Timeout!! Frame Number : {k} Not Received")
                    print("Retransmitting Window...")
                    break

            print()
            i += z
        except (BrokenPipeError, ConnectionResetError):
            print(f"Connection lost while sending frame {i}. Ending
transmission.")

            break

    return tt

def main():
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(('172.16.44.112', 9999))
    try:
        tf = int(input("Enter the Total number of frames: "))
        N = int(input("Enter the Window Size: "))
        tt = transmission(client_socket, N, tf)
        print(f"Total number of frames sent and resent: {tt}")
    finally:
        client_socket.close()
if __name__ == "__main__":
    main()
```

CLIENT CODE SCREENSHOT

```
1  import socket
2  import random
3  def transmission(client_socket, N, tf):
4      i = 1
5      tt = 0
6      while i <= tf:
7          z = 0
8          try:
9
10             for k in range(i, min(i + N, tf + 1)):
11                 msg = f"Frame {k}\n"
12
13                 client_socket.send(msg.encode())
14                 print(f"Sending {msg.strip()}...")
15                 tt += 1
16
17             for k in range(i, min(i + N, tf + 1)):
18                 ack = client_socket.recv(1024).decode()
19                 if ack == f"ACK {k}":
20                     print(f"Acknowledgment received for Frame {k}")
21                     z += 1
22                 else:
23                     print(f"Timeout!! Frame Number : {k} Not Received")
24                     print("Retransmitting Window...")
25                     break
26             print()
27             i += z
28
29         except (BrokenPipeError, ConnectionResetError):
30             print(f"Connection lost while sending frame {i}. Ending transmission.")
31             break
32     return tt
33
34 def main():
35     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
36     client_socket.connect(('172.16.44.112', 9999))
37     try:
38         tf = int(input("Enter the Total number of frames: "))
39         N = int(input("Enter the Window Size: "))
40         tt = transmission(client_socket, N, tf)
41         print(f"Total number of frames sent and resent: {tt}")
42     finally:
43         client_socket.close()
44
45 if __name__ == "__main__":
46     main()
```

OUTPUT

```
(base) karnwalpoorva23@poorvas-MacBook-Air ~ % python -u /Users/karnwalpoorva23/Desktop/hack/client.py
Enter the Total number of frames: 6
Enter the Window Size: 3
Sending Frame 1...
Sending Frame 2...
Sending Frame 3...
Acknowledgment received for Frame 1
Timeout!! Frame Number : 2 Not Received
Retransmitting Window...

Sending Frame 2...
Sending Frame 3...
Sending Frame 4...
Timeout!! Frame Number : 2 Not Received
Retransmitting Window...

Sending Frame 2...
Sending Frame 3...
Sending Frame 4...
Timeout!! Frame Number : 2 Not Received
Retransmitting Window...

Sending Frame 2...
Sending Frame 3...
Sending Frame 4...
Acknowledgment received for Frame 2
Acknowledgment received for Frame 3
Acknowledgment received for Frame 4

Sending Frame 5...
Sending Frame 6...
Timeout!! Frame Number : 5 Not Received
Retransmitting Window...

Sending Frame 5...
Sending Frame 6...
Timeout!! Frame Number : 5 Not Received
Retransmitting Window...

Sending Frame 5...
Sending Frame 6...
Acknowledgment received for Frame 5
Timeout!! Frame Number : 6 Not Received
Retransmitting Window...

Sending Frame 6...
Timeout!! Frame Number : 6 Not Received
Retransmitting Window...

Sending Frame 6...
Acknowledgment received for Frame 6

Total number of frames sent and resent: 20
(base) karnwalpoorva23@poorvas-MacBook-Air ~ %
```


QUESTION 3: SELECTIVE REPEAT ARQ

SERVER SIDE CODE:

```
import socket
import random
import time

class Receiver:
    def __init__(self, total_frames, port):
        self.total_frames = total_frames
        self.port = port
        self.acknowledged = [False] * total_frames
        self.received_frames = [None] * total_frames

    def start(self):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.bind(('localhost', self.port))
            print("Receiver ready and waiting for frames...")

            while True:
                data, addr = s.recvfrom(1024)
                frame_num = int(data.decode())

                if frame_num < 0 or frame_num >= self.total_frames:
                    continue

                self.receive(frame_num, s, addr)

    def receive(self, frame_num, s, addr):
        packet_loss_chance = random.random()
        if packet_loss_chance < 0.2:
            print(f"Packet {frame_num} lost.")
            return

        print(f"Received frame {frame_num} and buffered")
        self.received_frames[frame_num] = f"Frame {frame_num} data"
        self.acknowledge(frame_num, s, addr)

    def acknowledge(self, frame_num, s, addr):
        ack_loss_chance = random.random()
        if ack_loss_chance < 0.1:
            print(f"ACK for frame {frame_num} lost.")
            return

        print(f"ACK for frame {frame_num} sent")
        s.sendto(f"ACK {frame_num}".encode(), addr)

if __name__ == "__main__":
    total_frames = int(input("Enter total number of frames: "))
    port = 12345
    receiver = Receiver(total_frames, port)
    receiver.start()
```

CODE SCREENSHOT

```
server.py > Receiver > __init__
1  import socket
2  import random
3  import time
4  class Receiver:
5      def __init__(self, total_frames, port):
6          self.total_frames = total_frames
7          self.port = port
8          self.acknowledged = [False] * total_frames
9          self.received_frames = [None] * total_frames
10
11     def start(self):
12         with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
13             s.bind(('localhost', self.port))
14             print("Receiver ready and waiting for frames...")
15             while True:
16                 data, addr = s.recvfrom(1024)
17                 frame_num = int(data.decode())
18                 if frame_num < 0 or frame_num >= self.total_frames:
19                     continue
20                 self.receive(frame_num, s, addr)
21
22     def receive(self, frame_num, s, addr):
23         packet_loss_chance = random.random()
24         if packet_loss_chance < 0.2:
25             print(f"Packet {frame_num} lost.")
26             return
27         print(f"Received frame {frame_num} and buffered")
28         self.received_frames[frame_num] = f"Frame {frame_num} data"
29         self.acknowledge(frame_num, s, addr)
30
31     def acknowledge(self, frame_num, s, addr):
32         ack_loss_chance = random.random()
33         if ack_loss_chance < 0.1:
34             print(f"ACK for frame {frame_num} lost.")
35             return
36         print(f"ACK for frame {frame_num} sent")
37         s.sendto(f"ACK {frame_num}".encode(), addr)
38
39 if __name__ == "__main__":
40     total_frames = int(input("Enter total number of frames: "))
41     port = 12345
42     receiver = Receiver(total_frames, port)
43     receiver.start()
```

OUTPUT

```
ck/server.py"
Enter total number of frames: 6
Receiver ready and waiting for frames...
Received frame 0 and buffered
ACK for frame 0 sent
Received frame 1 and buffered
ACK for frame 1 sent
Received frame 2 and buffered
ACK for frame 2 sent
Packet 3 lost.
Received frame 4 and buffered
ACK for frame 4 lost.
Packet 5 lost.
Received frame 3 and buffered
ACK for frame 3 sent
Received frame 4 and buffered
ACK for frame 4 sent
Received frame 5 and buffered
ACK for frame 5 lost.
Packet 5 lost.
Received frame 5 and buffered
ACK for frame 5 sent
```

```
import socket
import time
class Sender:
```

```

import socket
import time
class Sender:
    def __init__(self, total_frames, window_size, timeout, port):

        self.total_frames = total_frames
        self.window_size = window_size
        self.timeout = timeout
        self.port = port
        self.frames = [False] * total_frames
        self.send_base = 0
        self.next_seq_num = 0
        self.timers = [None] * total_frames

    def start(self):

        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:

            receiver_addr = ('localhost', self.port)
            print("Sender started...")

            while self.send_base < self.total_frames:

                while self.next_seq_num < self.send_base + self.window_size and
self.next_seq_num < self.total_frames:

                    if not self.frames[self.next_seq_num]:

                        print(f"Sending frame {self.next_seq_num}")

                        s.sendto(str(self.next_seq_num).encode(), receiver_addr)
                        self.start_timer(self.next_seq_num)
                        self.next_seq_num += 1

                while True:

                    all_acks_received = True
                    start_time = time.time()
                    while time.time() - start_time < self.timeout:
                        try:

                            s.settimeout(self.timeout)
                            data, _ = s.recvfrom(1024)
                            ack_msg = data.decode()

                            if ack_msg.startswith("ACK"):

                                frame_num = int(ack_msg.split()[1])
                                if frame_num < len(self.frames):
                                    self.frames[frame_num] = True
                                    print(f"Frame {frame_num} acknowledged")
                                    self.stop_timer(frame_num)
                        except socket.timeout:

                            pass
                    resend_needed = False
                    for i in range(self.send_base, min(self.send_base +
self.window_size, self.total_frames)):

                        if not self.frames[i]:
                            resend_needed = True
                            print(f"Timeout for frame {i}, resending...")
                            s.sendto(str(i).encode(), receiver_addr)
                            self.start_timer(i)

```

```

        if not resend_needed:
            break

        self.slide_window()

    def slide_window(self):
        all_ack_received = True
        for i in range(self.send_base, min(self.send_base + self.window_size,
self.total_frames)):
            if not self.frames[i]:
                all_ack_received = False
                break

        if all_ack_received:
            self.send_base = min(self.send_base + self.window_size,
self.total_frames)
            self.next_seq_num = self.send_base
            print(f"Window slid. New base: {self.send_base}")
            if self.send_base < self.total_frames:
                while self.next_seq_num < self.send_base + self.window_size and
self.next_seq_num < self.total_frames:

                    if not self.frames[self.next_seq_num]:
                        print(f"Sending frame {self.next_seq_num}")
                        self.start_timer(self.next_seq_num)
                        self.next_seq_num += 1
    def start_timer(self, frame_num):
        self.timers[frame_num] = time.time()
    def stop_timer(self, frame_num):
        self.timers[frame_num] = None
if __name__ == "__main__":

    try:

        total_frames = int(input("Enter total number of frames: "))
        window_size = int(input("Enter window size: "))
        timeout = 3
        port = 12345
        if window_size <= 0 or total_frames <= 0:
            raise ValueError("Number of frames and window size must be positive
integers.")
        sender = Sender(total_frames, window_size, timeout, port)
        sender.start()

    except ValueError as e:

        print(f"Invalid input: {e}")

def main():

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(('172.16.44.112', 9999))
    try:

        tf = int(input("Enter the Total number of frames: "))
        N = int(input("Enter the Window Size: "))
        tt = transmission(client_socket, N, tf)
        print(f"Total number of frames sent and resent: {tt}")
    finally:
        client_socket.close()

if __name__ == "__main__":
    main()

```

CODE SCREENSHOT

Users > karnwalpoorva23 > Desktop > hack > client.py > Sender > start

```
1  import socket
2  import time
3  class Sender:
4      def __init__(self, total_frames, window_size, timeout, port):
5          self.total_frames = total_frames
6          self.window_size = window_size
7          self.timeout = timeout
8          self.port = port
9          self.frames = [False] * total_frames
10         self.send_base = 0
11         self.next_seq_num = 0
12         self.timers = [None] * total_frames
13     def start(self):
14         with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
15             receiver_addr = ('localhost', self.port)
16             print("Sender started...")
17             while self.send_base < self.total_frames:
18                 while self.next_seq_num < self.send_base + self.window_size and self.next_seq_num < self.total_frames:
19                     if not self.frames[self.next_seq_num]:
20                         print(f"Sending frame {self.next_seq_num}")
21                         s.sendto(str(self.next_seq_num).encode(), receiver_addr)
22                         self.start_timer(self.next_seq_num)
23                         self.next_seq_num += 1
24                 while True:
25                     all_acks_received = True
26                     start_time = time.time()
27                     while time.time() - start_time < self.timeout:
28                         try:
29                             s.settimeout(self.timeout)
30                             data, _ = s.recvfrom(1024)
31                             ack_msg = data.decode()
32                             if ack_msg.startswith("ACK"):
33                                 frame_num = int(ack_msg.split()[1])
34                                 if frame_num < len(self.frames):
35                                     self.frames[frame_num] = True
36                                     print(f"Frame {(variable) frame_num: int} received")
37                                     self.stop_timer(frame_num)
38                         except socket.timeout:
39                             pass
40                     resend_needed = False
41                     for i in range(self.send_base, min(self.send_base + self.window_size, self.total_frames)):
42                         if not self.frames[i]:
43                             resend_needed = True
44                             print(f"Timeout for frame {i}, resending...")
45                             s.sendto(str(i).encode(), receiver_addr)
46                             self.start_timer(i)
47                     if not resend_needed:
48                         break
49                 self.slide_window()
50
51     def slide_window(self):
52         all_ack_received = True
53         for i in range(self.send_base, min(self.send_base + self.window_size, self.total_frames)):
54             if not self.frames[i]:
55                 all_ack_received = False
56                 break
57
58         if all_ack_received:
59             self.send_base = min(self.send_base + self.window_size, self.total_frames)
60             self.next_seq_num = self.send_base
61             print(f"Window slid. New base: {self.send_base}")
62             if self.send_base < self.total_frames:
63                 while self.next_seq_num < self.send_base + self.window_size and self.next_seq_num < self.total_frames:
64                     if not self.frames[self.next_seq_num]:
65                         print(f"Sending frame {self.next_seq_num}")
66                         self.start_timer(self.next_seq_num)
67                         self.next_seq_num += 1
68
69     def start_timer(self, frame_num):
70         self.timers[frame_num] = time.time()
71     def stop_timer(self, frame_num):
72         self.timers[frame_num] = None
73 if __name__ == "__main__":
74     try:
75         total_frames = int(input("Enter total number of frames: "))
76         window_size = int(input("Enter window size: "))
77         timeout = 3
78         port = 12345
79         if window_size <= 0 or total_frames <= 0:
80             raise ValueError("Number of frames and window size must be positive integers.")
81         sender = Sender(total_frames, window_size, timeout, port)
82         sender.start()
```

```

83 |
84 |     except ValueError as e:
85 |         print(f"Invalid input: {e}")
86 |
87 | def main():
88 |     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
89 |     client_socket.connect(('172.16.44.112', 9999))
90 |     try:
91 |         tf = int(input("Enter the Total number of frames: "))
92 |         N = int(input("Enter the Window Size: "))
93 |         tt = transmission(client_socket, N, tf)
94 |         print(f"Total number of frames sent and resent: {tt}")
95 |     finally:
96 |         client_socket.close()
97 | if __name__ == "__main__":
98 |     main()

```

OUTPUT

```

ack/client.py"
Enter total number of frames: 6
Enter window size: 3
Sender started...
Sending frame 0
Sending frame 1
Sending frame 2
Frame 0 acknowledged
Frame 1 acknowledged
Frame 2 acknowledged
Window slid. New base: 3
Sending frame 3
Sending frame 4
Sending frame 5
Timeout for frame 3, resending...
Timeout for frame 4, resending...
Timeout for frame 5, resending...
Timeout for frame 3, resending...
Timeout for frame 4, resending...
Timeout for frame 5, resending...
Frame 3 acknowledged
Frame 4 acknowledged
Timeout for frame 5, resending...
Timeout for frame 5, resending...
Frame 5 acknowledged
Window slid. New base: 6

```